2ip90
Arrays

arrays
for-each loop
Chapter 10

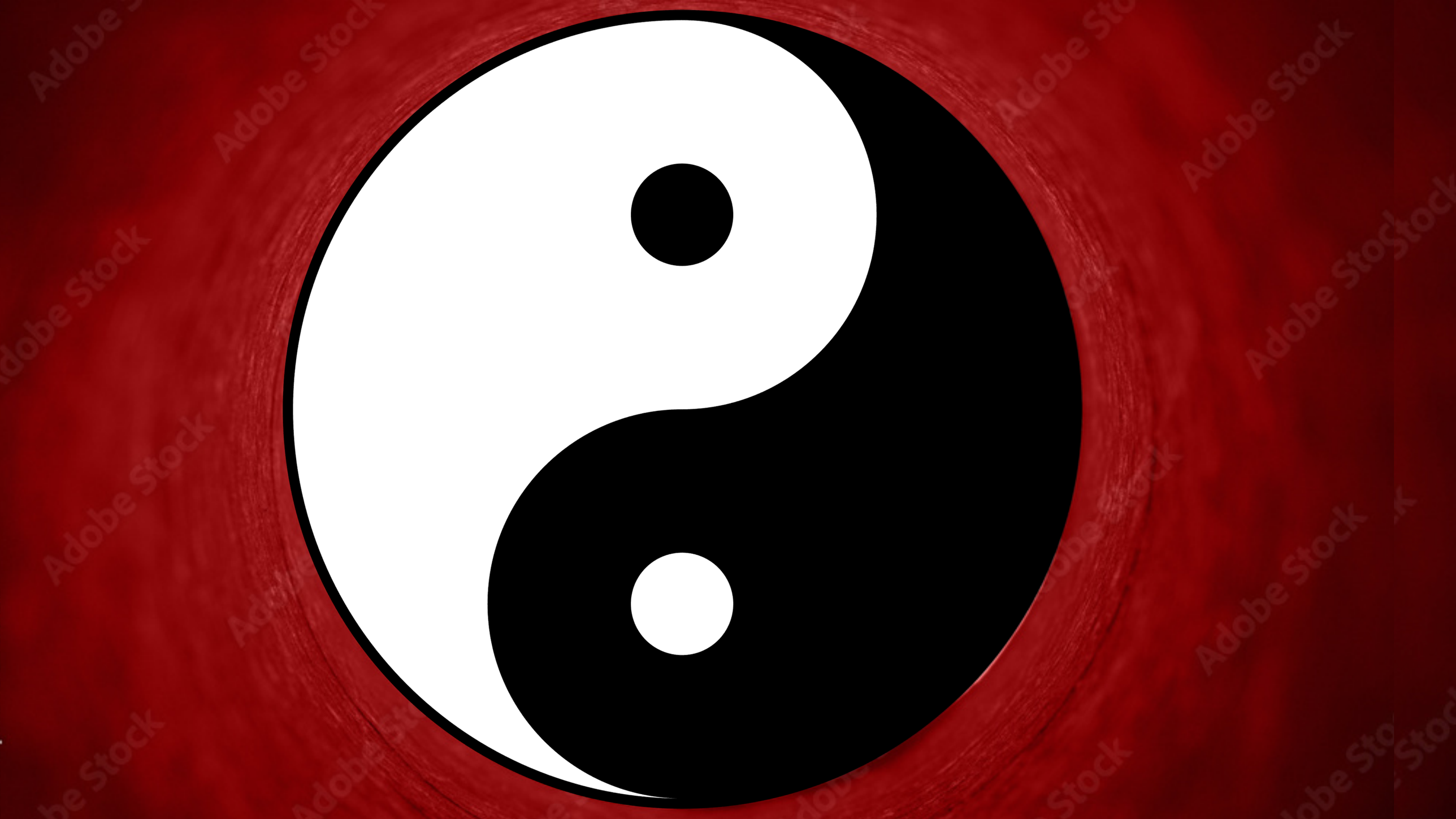


control and data
the yin and yang of programming

- Two fundamental concepts of programming are control and data. Data are values stored in variables. Control determines what is done with the data.
- A program executes the statements (lines or parts of it) one after the other. The order (called control) is determined by:

  1. the order in the text
  2. control structuring

- We have seen some ways to structure control: make while-loops, make if-structures



Simple structuring of data: arrays

- An array is a sequence of numbered variables

- Instead of

```
int student1;
int student2;
int student3;

student1 = 7;
student2 = 5;
student3 = 3;
```

- you can do:

```
int[] students;

students = new int[3];
students[0] = 7;
students[1] = 5;
students[2] = 3;
```

declaration

creation: gives 3 elements

last one is `students[2]`

Structuring of data: arrays

- An array is a sequence of numbered variables

- Instead of

```
int student1;
int student2;
int student3;

student1 = 7;
student2 = 5;
student3 = 3;
```

- you can do:

```
int[] students;

students = new int[3];
students[0] = 7;
students[1] = 5;
students[2] = 3;
```

declaration

creation: gives 3 elements

last one is `students[2]`

Arrays

- Number between [] is called index
- In Java, all arrays start with 0. Last index is the number of elements - 1
- Declaration and creation are separate; often you don't know immediately how many variables you need, e.g., asked as input from the user
- declaration and creation can be combined, though:

```
int[] students = new int[3];
```

- after creation, size is fixed
  - only possibility to "change" size is to create a new array (which will be empty)
- attempt to access outside the bounds: error
  - in Java called an exception; can make your program crash

Arrays - example

```
int[] students;
double sum;  // collects the sum (accumulator)
int i;       // index variable

students = new int[3];
students[0] = 7;
students[1] = 5;
students[2] = 6;

// calculate sum
sum = 0;
i = 0;
while (i < 3) {
   sum = sum + students[i];
  i = i + 1;
}
```

Intermezzo

official term:
computable

- Now you can write anything that is programmable at all
- that is, any computation or algorithm (method) that can be precisely described
- Java with arrays and loops is equivalent in power to so-called Turing machines
- any computation that can be precisely described can be described by a Turing machine (Church-Turing thesis)
- since "any computation..." can not be formally defined, this is a conjecture (not a theorem)

for loop - example

```
int[] students;


students = new int[3];
students[0] = 7;
students[1] = 5;
students[2] = 6;

// calcualte sum
double sum = 0.0;

 int i = 0;
 while (i<3){
   sum = sum + students[i];
  i++;
 }

double average = sum/3;
```

Flexibility - example

Using the number 3 is inflexible
Make the program work for any number of students

```
System.out.println("Give number of students: ");
int n = scanner.nextInt();

int[] students = new int[n];
for (int i=0; i<n; i=i+1) {
    students[i] = scanner.nextInt();
}


// calculate sum
double sum = 0.0;
for (int i=0; i< n; i=i+1) {
    sum = sum + students[i];
}
double average = sum/n;
```



beware: division does not work
when sum and n are both ints

using length (size) of arrays

- following program works for students array of any length does not depend on a variable you have to maintain

```
System.out.println("The following students are below average:");
for (int i=0; i<students.length; i=i+1) {
  if (students[i] < average) {
    System.out.println(i);
  }
}
```

- Advise: use variables or .length instead of fixed numbers

Improvement - example

students.length gives you the number of elements in the array

works safer than the variable n

Note: trying to access an array element that doesn't exist is a serious error

```
System.out.println("Give number of students: ");
int n = scanner.nextInt();

int[] students = new int[n];
for (int i=0; i < students.length; i=i+1) {
  students[i] = scanner.nextInt();
}

// calculate sum
double sum = 0.0;
for (int i=0; i< students.length; i=i+1) {
   sum = sum + students[i];
}
double average = sum/students.length;
```

This works for any number of studensts; the input has to be adapted

Quiz 1

- Given:
```
int[] a = new int[3];
int n;
```
- What is correct Java? (not necessarily true)

multiple answers

A. `n = a[3];`

B. `a.length == a[0];`

C. `n = a[a[1]];`

D. `a[-1] = n;`

Quiz 1

- Given:

```
int[] a = new int[3];
int n;
```

- What is correct Java? (not necessarily true)

multiple answers

A. `n = a[3];` — INCORRECT because there is no index 3

B. `a.length == a[0];` — CORRECT because there is an index 0 (it is false, btw)

C. `n = a[a[1]];` — CORRECT because a[1] is 0

D. `a[-1] = n;` — INCORRECT because there is no index -1

Quiz 2

```
int a[] = new int[4];
for (int i = 0; i < a.length; i++) {
  a[i] = i + 2;
}

int s = 0;
for (int i = a.length-1; i > 0; i--) {
  s = s + a[i];
}

// s?
```

- What is s at the end?

A. 8

B. 10

C. 12

D. 14

```
Note: all these three statements do the same:
i = i - 1
i -= 1
i--
```

Quiz 2

answer

for-each loop

- common need: each element of an array (or another collection) has to be visited
- for-each loop gives exactly this without need for an index variable
- for-each works for all kinds of collections, such as lists, sets, trees (will be encountered later)

for-each loop — example

```
int[] grades = new int[3];
grades[0] = 7;
grades[1] = 5;
grades[2] = 6;

// calculate sum
double sum = 0.0;

for (int i=0; i < grades.length; i++) {
  sum = sum + grades[i];
}

double average = sum/3;
```

read: "for each grade in grades”

assign-with-operation

aside

- "plus and becomes"

```
sum += b;
```

```
sum = sum + b;
```

- other:

```
prod *= q;
n /= 2;   (halving)
i %= a.length
```

```
prod = prod * q;
n = n / 2;
i = i % a.length
```

for-each loop — syntax

```
for ( <type> <variable> : <collection> ) {
    <statements>
}
```

- <type>: member (element) type of collection
- <collection>: array or other collection with members of type <type>
- <var>: variable that will contain the members of the collection

ArrayList and others

more about this later

Javadoc

- Tool to generate HTML documents out of source code
- Tags @... are recognized in comment that starts with /**
- What comes after the tag is treated as its value
- Examples:

```
/** @author Drs. P
 *  @date   12 Sep 2024
 */
```

- Some tools support @inv or @invariant

Size does matter

- `length`
  number of elements in an array

- `length()`
  number of characters in a String

- `size()`
  number of elements in an ArrayList (and other Collections)